# University of Moratuwa

## Faculty of Engineering




## EN3150 – Pattern Recognition

# Assignment 03

## Resource-Constrained CNN for Edge Image Classification

Group: **GalTricals**

### Group Members

| Name | Index Number |
|---|---|
| CHINTHAKA M.K.M.G. | 230105H |
| DILMITH G.S. | 230149U |
| KAUSHAL S.V. | 230326K |
| RAJAPAKSHA J.S.W. | 230511A |

**Abstract**

This report designs, trains and evaluates convolutional neural networks (CNNs) for an extreme-edge use case: classifying five rice varieties (Arborio, Basmati, Ipsala, Jasmine, Karacadag) on a low-resource sorting machine. All 75,000 images of the Kaggle Rice Image Dataset were down-scaled to $64 \times 64 \times 3$ and split 70/15/15 (stratified), giving 52,500 / 11,250 / 11,250 images. We built a standard CNN (Model A, 1,142,597 parameters) and a depthwise-separable CNN (Model B, 79,168 parameters, below the 100,000 limit), compared five optimizers, and benchmarked both models against fine-tuned MobileNetV2 and EfficientNetB0.

On the held-out test set Model A reaches **99.55%** accuracy, Model B **97.99%**, MobileNetV2 **98.41%** and EfficientNetB0 **99.45%**. Model B needs 14× fewer parameters than Model A and 30× fewer than MobileNetV2; its saved file is 994 KB (about 94% smaller than EfficientNetB0's) and its forward pass costs about 7.6 M multiply-accumulates (MACs) against 25–33 M for the pre-trained networks. The price is a 4.4× higher test error than Model A (2.01% vs. 0.45%), concentrated in two class pairs. Of the five optimizers only Adam trained at the shared learning rate of $10^{-3}$. We also document the limits of these results: single runs, a frozen SOTA backbone, optimizer state included in the saved file sizes, and activation memory that is not captured by parameter count.

# Contents

**1 Introduction and Objectives** **4**

**2 Dataset and Pre-processing Pipeline** **4**

2.1 Dataset . . . 4

2.2 Pipeline and split . . . 4

**3 Custom Architecture Design** **6**

3.1 Activation functions: hardware-aware justification . . . 6

3.2 Model A: standard CNN . . . 6

3.3 Model B: depthwise-separable CNN . . . 7

3.4 Computational cost (multiply-accumulates) . . . 8

**4 Optimizer Selection and Tuning** **9**

4.1 Candidates . . . 9

4.2 Experiment . . . 9

4.3 Interpretation . . . 10

**5 Custom Model Training and Evaluation** **10**

5.1 Set-up . . . 10

5.2 Loss and accuracy curves . . . 13

5.3 Test-set evaluation . . . 14

5.4 Model comparison: standard vs. depthwise-separable . . . 15

**6 Comparison with Lightweight State-of-the-Art Networks** **15**

6.1 Transfer-learning set-up . . . 15

6.2 Training dynamics . . . . . 16
6.3 Test-set results . . . . . 17

**7 Final Comparison and Trade-off Discussion** **18**
7.1 Accuracy versus memory footprint . . . . . 18
7.2 Computational cost and latency . . . . . 19
7.3 Activation memory: weights are not the whole story . . . . . 19
7.4 Advantages and limitations of a custom sub-100k model . . . . . 20

**8 Limitations and Threats to Validity** **20**

**9 Conclusion and Future Work** **20**

**10 Source Code and Reproducibility** **21**

# List of Figures


1 Pre-processing and splitting pipeline used for the final experiments. . . . . 5
2 Sample training images (64×64) . . . . . 6
3 Model A architecture (1,142,597 parameters). . . . . 8
4 Model B architecture (79,168 parameters). Each SepConv is a $3 \times 3$ depthwise followed by a $1 \times 1$ pointwise convolution; all convolutions use "same" padding. . . 8
5 Parameters and forward-pass MACs for the four models (TensorFlow profiler, $64 \times 64 \times 3$ input, untrained weights). . . . . 9
6 Validation loss for the five optimizers . . . . . 11
7 Validation accuracy for the five optimizers. Only Adam leaves chance level (20%). 12
8 Training and validation loss over 20 epochs. . . . . 13
9 Training and validation accuracy, Models A and B . . . . . 13
10 Confusion matrices on the test set (rows: actual class, columns: predicted class). . 14
11 Training (solid) and validation (dashed) loss of the fine-tuned SOTA models. . . . 16
12 Training (solid) and validation (dashed) accuracy of the fine-tuned SOTA models. 17
13 Test accuracy vs. saved model size . . . . . 18
14 Average training time per epoch . . . . . 19


# List of Tables


1 Summary of dataset properties. . . . . 5
2 Model A layer specification and parameter count . . . . . 7
3 Model B layer specification and parameter count . . . . . 7
4 Analytic per-layer MACs, standard vs. depthwise-separable convolution (bias, ReLU and pooling excluded). . . . . 8
5 Optimizer comparison on Model B . . . . . 10
6 Per-class test metrics, Models A and B . . . . . 14
7 Model A vs. Model B. MB = $1024^2$ bytes, KB = 1024 bytes. . . . . 15
8 Parameter counts of the fine-tuned SOTA models. . . . . 16
9 Per-class test metrics, SOTA models . . . . . 17
10 Final comparison on the 11,250-image test set. MB = $1024^2$ bytes. . . . . 18
11 Custom lightweight model: advantages and limitations. . . . . 20

# 1 Introduction and Objectives

Deploying deep networks on microcontrollers, Raspberry Pi-class boards and other edge nodes is constrained by memory, compute and power rather than by accuracy alone. The assignment asks for a lightweight image classifier for such a device. Our target application is an *automated agricultural sorting machine* that classifies individual rice grains in real time.

The objectives, following the assignment brief, are:

1. prepare a low-resolution ($\leq 64 \times 64$) dataset with a 70/15/15 split (Section 2);
2. design a standard CNN (Model A) and a depthwise-separable CNN (Model B, $< 100{,}000$ trainable parameters), with explicit parameter counts and hardware-aware activation choices (Section 3);
3. select and justify an optimizer and learning rate, comparing against SGD and SGD with momentum (Section 4);
4. train both models for at least 20 epochs and report loss curves, accuracy, confusion matrices, precision/recall, size and training time (Section 5);
5. fine-tune two lightweight state-of-the-art (SOTA) networks on the same splits (Section 6);
6. discuss accuracy, memory and computational trade-offs (Sections 7–8).

# 2 Dataset and Pre-processing Pipeline

## 2.1 Dataset

We use the Kaggle *Rice Image Dataset* [3, 4]: colour photographs of single rice grains on a black background, in five classes – Arborio, Basmati, Ipsala, Jasmine and Karacadag – with exactly 15,000 images per class (75,000 in total). The classes differ in grain length, width, aspect ratio, colour and surface texture, which are the cues a small CNN can learn at low resolution. The equal class sizes mean no class re-weighting is needed.

The final results in this report use the **complete dataset**. (The notebook also contains an optional "Code Block-2" that sub-samples 2,000 images per class for memory-limited machines; it is not used for any number reported here.)

## 2.2 Pipeline and split

Figure 1 shows the pipeline and Table 1 summarises the data. File paths and labels are collected first, then split with scikit-learn's stratified `train_test_split` (70% train; the remaining 30% divided 50/50 into validation and test; `random_state`=42). Splitting paths before any image is loaded rules out leakage between the sets. Each image is then decoded and resized to $64 \times 64 \times 3$ with `tf.image.resize` (bilinear), batched (32) and prefetched. Pixel scaling is done *inside* each network so that the exported model accepts raw $[0, 255]$ pixels: Model A and B use `Rescaling(1/255)` $\rightarrow [0, 1]$; MobileNetV2 uses `Rescaling(1/127.5, offset=-1)` $\rightarrow [-1, 1]$; EfficientNetB0 receives raw $[0, 255]$ because it normalises internally. All Python, NumPy and TensorFlow seeds are fixed to 42.

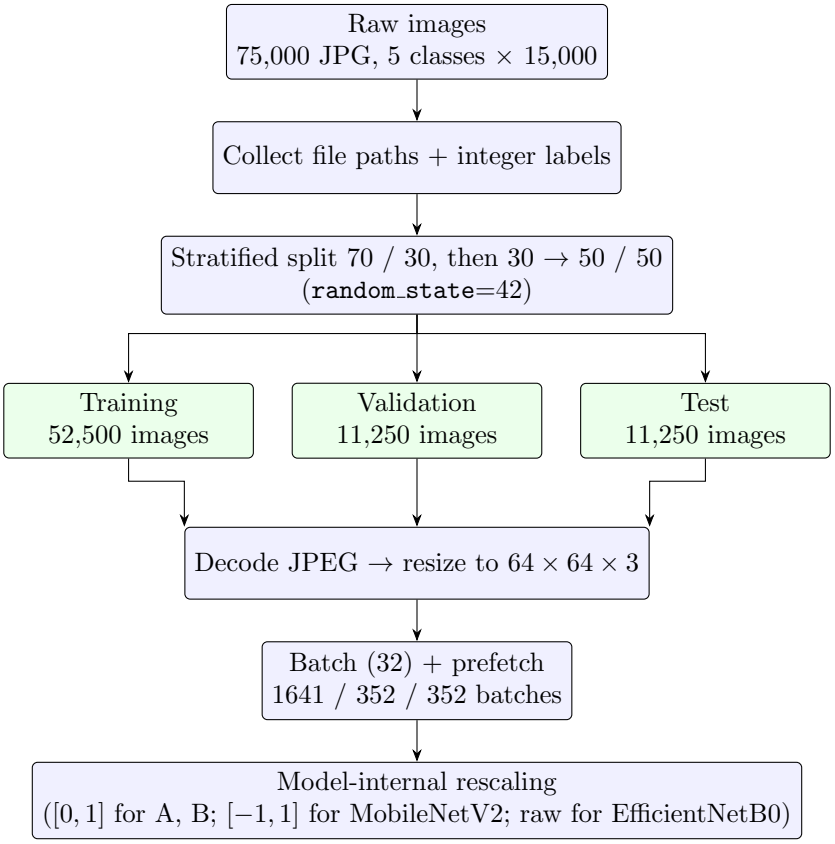



**Figure 1:** Pre-processing and splitting pipeline used for the final experiments.

| **Property** | **Value** |
|---|---|
| Dataset | Rice Image Dataset (Kaggle) |
| Task | Five-class rice variety classification |
| Images per class / total | 15,000 / 75,000 (full dataset) |
| Input resolution | $64 \times 64 \times 3$ (RGB) |
| Train / validation / test split | 70% / 15% / 15% (stratified) |
| Images per split | 52,500 / 11,250 / 11,250 |
| Test images per class | 2,250 |
| Batch size and batches per epoch | 32; 1641 train, 352 validation, 352 test |

**Table 1:** Summary of dataset properties.

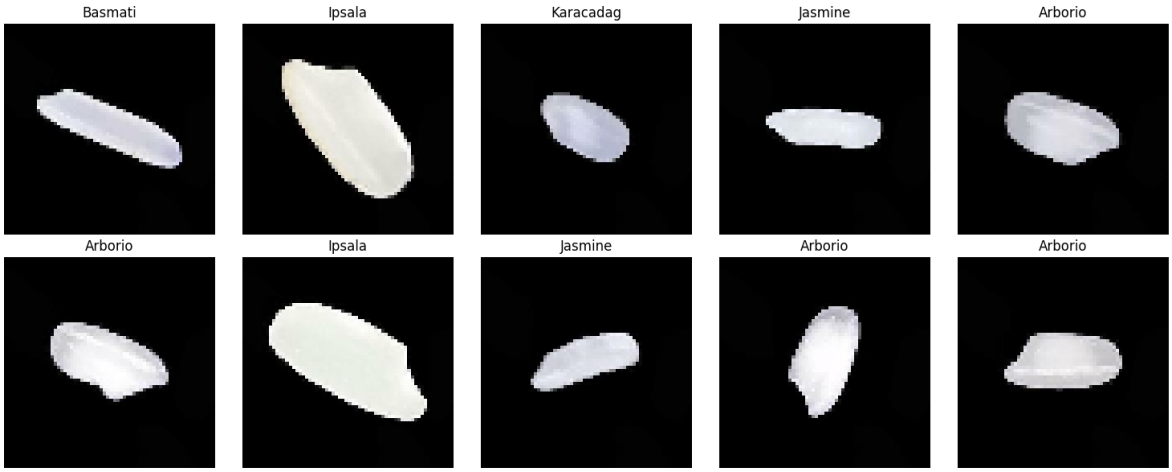



**Figure 2:** Ten random training images after down-scaling to $64 \times 64$. Ipsala grains are visibly the largest; Basmati and Jasmine are both elongated; Arborio and Karacadag are short and rounded – a hint of where confusions will occur (Section 5).

# 3 Custom Architecture Design

## 3.1 Activation functions: hardware-aware justification

Sigmoid and tanh saturate at their tails, so their derivatives approach zero and the gradients reaching early layers vanish [1]. All hidden layers in both models therefore use the Rectified Linear Unit,

$$f(x) = \max(0, x), \qquad f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases} \tag{1}$$

which has a constant gradient of one for positive inputs. For edge hardware ReLU has further advantages:

- it is a single comparison/selection – no exponential, division or look-up table, unlike sigmoid and tanh;
- it works unchanged in integer arithmetic, so it survives INT8 quantisation (Section 9);
- max-pooling (comparisons only) and global average pooling (additions and one division, zero parameters) are likewise cheap.

The output layer uses softmax, which needs exponentials, but it is only needed to produce probabilities for the loss. Softmax is monotonic, so on the device the arg-max of the five logits gives the same class and the exponentials can be dropped at inference.

## 3.2 Model A: standard CNN

Model A is the high-capacity baseline (cf. the CNN topology in [2]): three blocks of $3 \times 3$ `Conv2D` ("same" padding, ReLU) each followed by $2 \times 2$ max-pooling, then flatten, a 128-unit dense layer and a 5-way softmax (Table 2, Figure 3). A standard convolution has

$$P_{\text{conv}} = (K_h K_w C_{in} + 1)\, C_{out}. \tag{2}$$

| Layer | Output shape | Operation | Act. | Parameters |
|---|---|---|---|---|
| Input + Rescaling | (64, 64, 3) | pixels $\times 1/255$ | – | 0 |
| Conv2D 1 | (64, 64, 32) | $3 \times 3$, 32 filters | ReLU | $(27+1) \cdot 32 = 896$ |
| MaxPool 1 | (32, 32, 32) | $2 \times 2$ | – | 0 |
| Conv2D 2 | (32, 32, 64) | $3 \times 3$, 64 filters | ReLU | $(288+1) \cdot 64 = 18{,}496$ |
| MaxPool 2 | (16, 16, 64) | $2 \times 2$ | – | 0 |
| Conv2D 3 | (16, 16, 128) | $3 \times 3$, 128 filters | ReLU | $(576+1) \cdot 128 = 73{,}856$ |
| MaxPool 3 | (8, 8, 128) | $2 \times 2$ | – | 0 |
| Flatten | (8192) | – | – | 0 |
| Dense 1 | (128) | fully connected | ReLU | $(8192+1) \cdot 128 = 1{,}048{,}704$ |
| Dense 2 (output) | (5) | fully connected | Softmax | $(128+1) \cdot 5 = 645$ |
| **Total trainable parameters** | | | | **1,142,597** |

**Table 2:** Model A layer-by-layer specification and parameter count (verified against `model.summary()`).

Note that 92% of Model A's parameters (1,048,704 of 1,142,597) sit in the first dense layer, a consequence of flattening an $8 \times 8 \times 128$ feature map.

### 3.3 Model B: depthwise-separable CNN

Model B replaces every standard convolution with a depthwise-separable one: a $3 \times 3$ depthwise filter applied to each input channel separately, followed by a $1 \times 1$ pointwise convolution that mixes channels [8, 9]. It adds a fourth block (256 filters) and replaces `Flatten` by **global average pooling** (GAP), which removes the 1.05 M-parameter dense layer. The parameter count of one separable layer is

$$P_{\text{sep}} = K_h K_w C_{in} + (C_{in} C_{out} + C_{out}). \tag{3}$$

| Layer | Output shape | Operation | Act. | Parameters |
|---|---|---|---|---|
| Input + Rescaling | (64, 64, 3) | pixels $\times 1/255$ | – | 0 |
| SepConv 1 | (64, 64, 32) | dw $3 \times 3$ + pw $1 \times 1$, 32 | ReLU | $27 + 128 = 155$ |
| MaxPool 1 | (32, 32, 32) | $2 \times 2$ | – | 0 |
| SepConv 2 | (32, 32, 64) | dw $3 \times 3$ + pw $1 \times 1$, 64 | ReLU | $288 + 2112 = 2{,}400$ |
| MaxPool 2 | (16, 16, 64) | $2 \times 2$ | – | 0 |
| SepConv 3 | (16, 16, 128) | dw $3 \times 3$ + pw $1 \times 1$, 128 | ReLU | $576 + 8320 = 8{,}896$ |
| MaxPool 3 | (8, 8, 128) | $2 \times 2$ | – | 0 |
| SepConv 4 | (8, 8, 256) | dw $3 \times 3$ + pw $1 \times 1$, 256 | ReLU | $1152 + 33{,}024 = 34{,}176$ |
| MaxPool 4 | (4, 4, 256) | $2 \times 2$ | – | 0 |
| GlobalAvgPool | (256) | spatial mean | – | 0 |
| Dense 1 | (128) | fully connected | ReLU | $(256+1) \cdot 128 = 32{,}896$ |
| Dense 2 (output) | (5) | fully connected | Softmax | $(128+1) \cdot 5 = 645$ |
| **Total trainable parameters** | | | | **79,168** ($< 100{,}000$) |

**Table 3:** Model B layer-by-layer specification and parameter count (verified against `model.summary()`). dw = depthwise, pw = pointwise.

For the first layer alone the parameters fall from 896 to 155 (82.7% fewer). Overall, Model B has **93.1% fewer parameters than Model A** (79,168 vs. 1,142,597) and 20.8% headroom below the 100,000 budget.

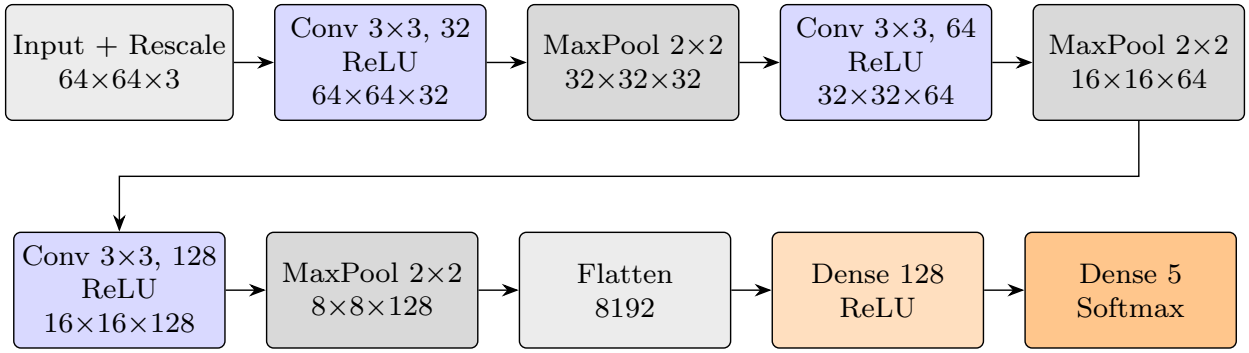



**Figure 3:** Model A architecture (1,142,597 parameters).

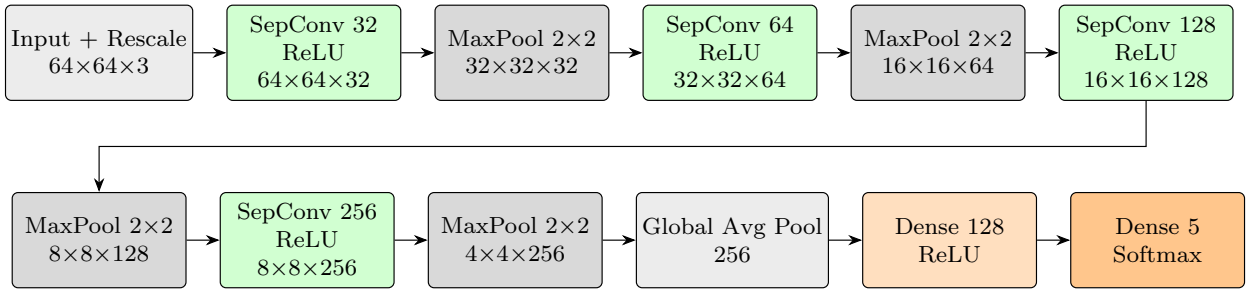



**Figure 4:** Model B architecture (79,168 parameters). Each SepConv is a $3 \times 3$ depthwise followed by a $1 \times 1$ pointwise convolution; all convolutions use "same" padding.

### 3.4 Computational cost (multiply-accumulates)

Parameter count measures memory; MACs measure compute. A standard convolution costs $HWK_hK_wC_{in}C_{out}$ MACs per layer; a separable one costs $HW(K_hK_wC_{in} + C_{in}C_{out})$, a ratio of $1/C_{out} + 1/(K_hK_w)$, close to 1/9 for wide layers. Table 4 confirms this for the three layers that exist in both networks.

| Layer (feature map) | Standard conv MACs | Separable MACs | Reduction |
|---|---|---|---|
| Layer 1 (64×64, 3 → 32) | 3,538,944 | 503,808 | 7.0× |
| Layer 2 (32×32, 32 → 64) | 18,874,368 | 2,392,064 | 7.9× |
| Layer 3 (16×16, 64 → 128) | 18,874,368 | 2,244,608 | 8.4× |

**Table 4:** Analytic per-layer MACs, standard vs. depthwise-separable convolution (bias, ReLU and pooling excluded).

For whole networks we count floating-point operations with the TensorFlow profiler on the untrained architectures (batch of one $64 \times 64 \times 3$ image; MACs $\approx$ FLOPs/2; this includes bias, activation and normalisation ops, so values are approximate). Model A needs about 42.6 M MACs, Model B 7.6 M (**5.6× fewer**), MobileNetV2 25.2 M and EfficientNetB0 33.5 M (Figure 5). Model B has 30× fewer parameters than MobileNetV2 but only about 3.3× fewer MACs, largely because MobileNetV2 is itself built from depthwise-separable convolutions; parameter count and compute are different quantities and should not be conflated.

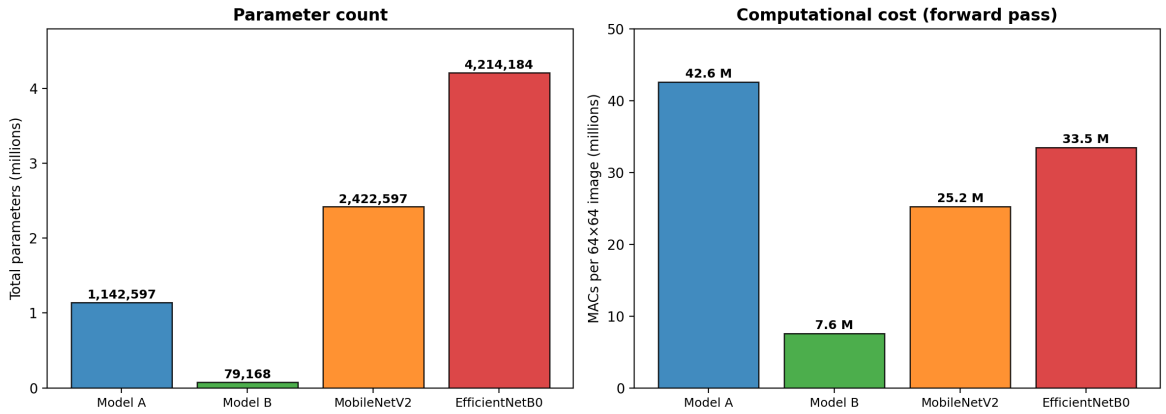



**Figure 5:** Parameters and forward-pass MACs for the four models (TensorFlow profiler, $64 \times 64 \times 3$ input, untrained weights).

# 4 Optimizer Selection and Tuning

## 4.1 Candidates

Training minimises sparse categorical cross-entropy. The weight update differs per optimizer ($\eta =$ learning rate, $g_t = \nabla_\theta J$):

- **SGD:** $\theta_t = \theta_{t-1} - \eta g_t$. Uses only the current mini-batch gradient; it oscillates across narrow valleys and slows on plateaus and saddle points.
- **SGD with momentum:** $v_t = \gamma v_{t-1} + \eta g_t$, $\theta_t = \theta_{t-1} - v_t$. The velocity accumulates consistent gradient directions (speed-up) and cancels alternating ones (damping). With $\gamma = 0.9$ the steady-state step along a consistent gradient is $\eta/(1-\gamma) = 10\eta$.
- **RMSprop** divides the step by a running RMS of recent gradients; **Adagrad** divides by the accumulated sum of squared gradients, so its effective step only ever shrinks.
- **Adam** [5] combines momentum (first-moment estimate) with RMSprop-style per-parameter scaling (second-moment estimate) and bias correction, so each weight receives a step of roughly $\eta$ regardless of its gradient scale.

## 4.2 Experiment

Each optimizer trained a freshly initialised copy of Model B for 15 epochs on the full training set, at the **same learning rate** $\eta = 10^{-3}$, batch size 32, with momentum $\gamma = 0.9$ for SGD+Momentum. Table 5 gives the final validation accuracy; Figures 6 and 7 show the curves (top panel: all optimizers; lower panels: each optimizer on its own axis, because the non-Adam curves are invisible on a shared axis).

| Optimizer | Final val. accuracy (epoch 15) | Final val. loss |
|---|---|---|
| SGD | 20.00% | $\approx 1.6094$ |
| SGD + Momentum ($\gamma = 0.9$) | 20.00% | $\approx 1.6094$ |
| RMSprop | 20.00% | $\approx 1.6095$ |
| Adagrad | 20.00% | $\approx 1.6093$ |
| **Adam** | **95.65%** | $\approx 0.14$ |

**Table 5:** Optimizer comparison on Model B ($\eta = 10^{-3}$, 15 epochs). 20% is chance level for five balanced classes and $\ln 5 \approx 1.6094$ is the cross-entropy of a uniform prediction.

### 4.3 Interpretation

**What the data show.** Adam reached 95–99% validation accuracy in every epoch except one (91.5% at epoch 6). The other four optimizers never left chance level: their validation loss stayed at $\ln 5$, i.e. the network kept predicting an almost uniform distribution. Their losses did decrease monotonically, but by only $\sim 10^{-5}$ over 15 epochs (SGD $\approx 2.5 \times 10^{-5}$, SGD+Momentum $\approx 3.0 \times 10^{-5}$, read from Figure 6); RMSprop's loss was flat at 1.60948 apart from one blip near epoch 11. Adam's validation curve is noisy – accuracy ranges from 91.5% (epoch 6) to 98.8% (epoch 14) – so the final-epoch value of 95.65% understates its typical level (the 20-epoch run in Section 5 ends at 97.7%).

**Impact of the momentum parameter.** In theory $\gamma$ multiplies the effective learning rate by up to $1/(1-\gamma)$ and damps oscillations across steep directions, which should speed up SGD. In our run, however, the effect is negligible: with $\eta = 10^{-3}$ the gradient steps are so small that even a ten-fold amplification ($\eta_{\text{eff}} \approx 10^{-2}$) moved the loss only marginally further than plain SGD ($\sim 3.0$ vs. $\sim 2.5 \times 10^{-5}$) and neither escaped the initial plateau. Momentum therefore did not change the final accuracy here. We cannot say from this experiment how momentum behaves at a well-tuned learning rate.

**Why the comparison favours Adam (a caveat).** A single shared $\eta = 10^{-3}$ is Adam's default but is far too small for plain SGD, which typically needs $10^{-2}$–$10^{-1}$ on networks like this. Adam's per-parameter normalisation makes it far less sensitive to the gradient scale, and this is why it works out of the box here, where the initial gradients are small and there is no batch normalisation. The result is therefore "at $\eta = 10^{-3}$ only Adam trains", not "SGD cannot train this model". A proper tuning study would sweep $\eta \in \{10^{-3}, 10^{-2}, 10^{-1}\}$ and $\gamma \in \{0, 0.5, 0.9, 0.99\}$ for SGD; we did not run it and list it as future work.

**Decision.** We trained the final models with **Adam,** $\eta = 10^{-3}$: it is the only optimizer that converged in our budget, it needs little tuning, and it has a low per-step cost. Its drawbacks are two extra state tensors per weight (relevant for the saved file sizes, Section 5.4) and a noisy validation curve, which a learning-rate decay schedule or best-epoch checkpointing would address.

## 5 Custom Model Training and Evaluation

### 5.1 Set-up

Both models were trained for **20 epochs** with Adam ($\eta = 10^{-3}$), batch size 32, on a Google Colab GPU, using the identical train/validation split. Per-epoch training time is the wall-clock time of the whole `fit` call divided by 20 (so it includes validation).

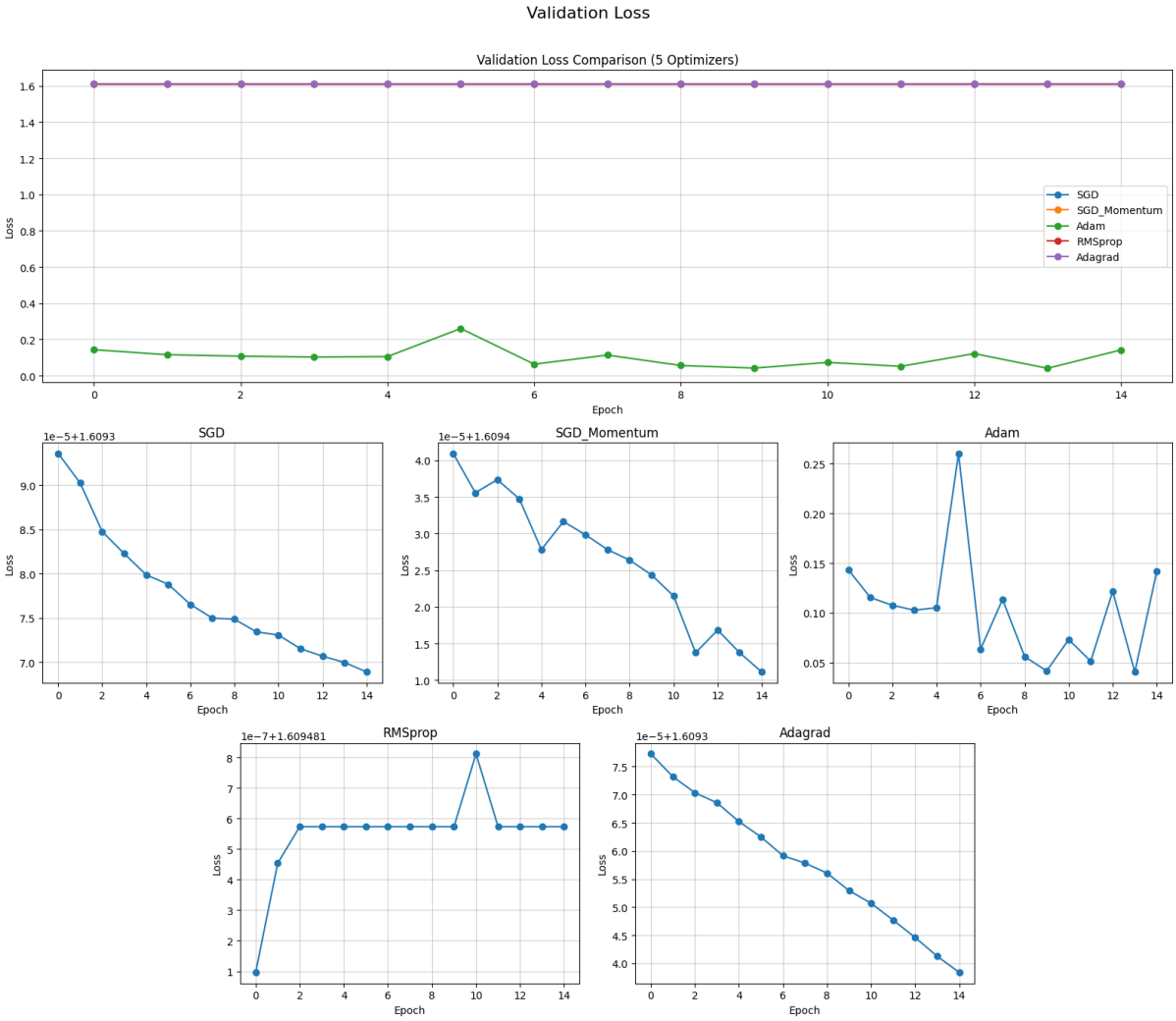



**Figure 6:** Validation loss for the five optimizers. Top: combined plot. Bottom: individual plots (note the very different $y$-axis scales; the offsets such as $10^{-5} + 1.6093$ show how little the non-Adam losses move).

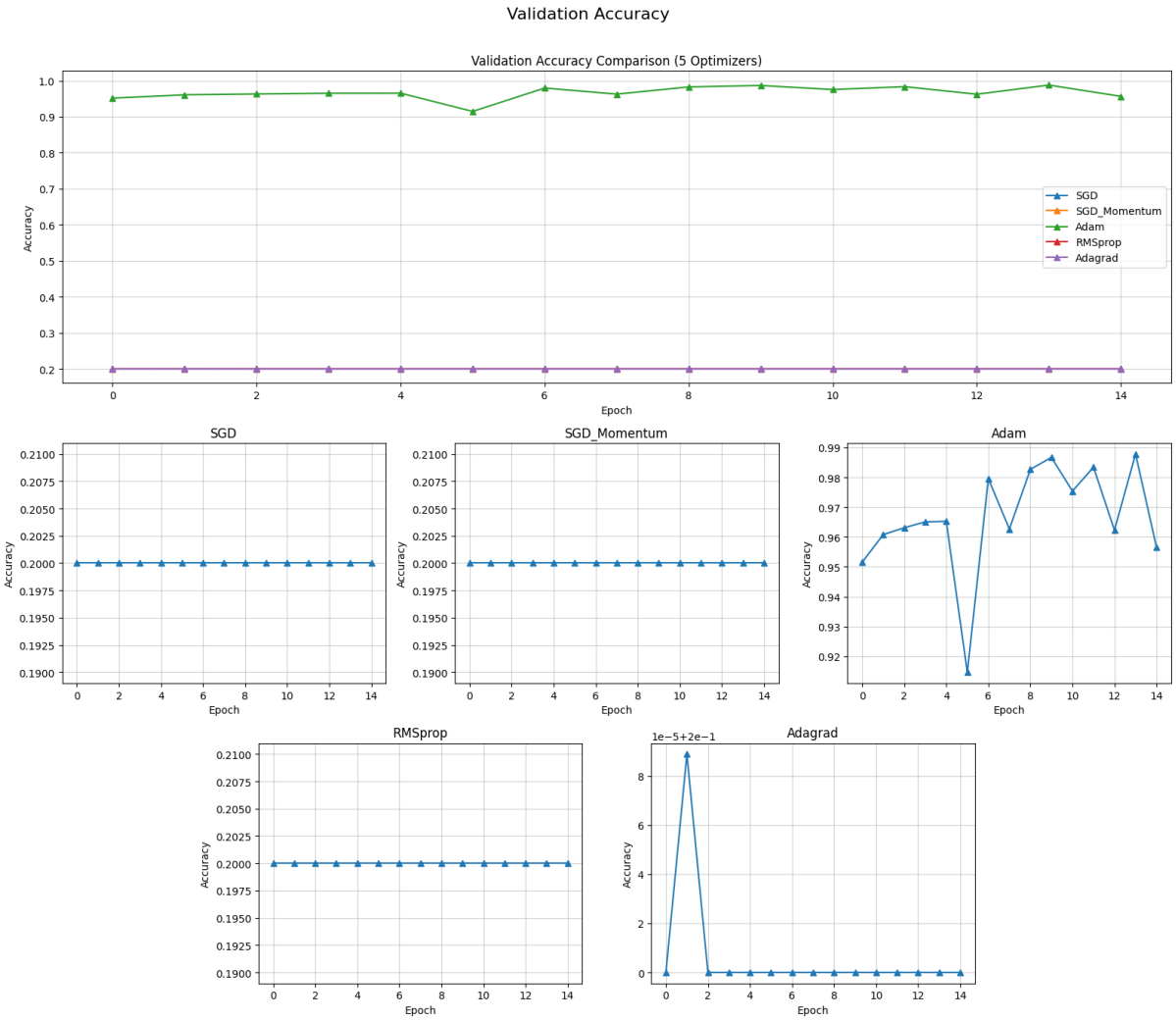
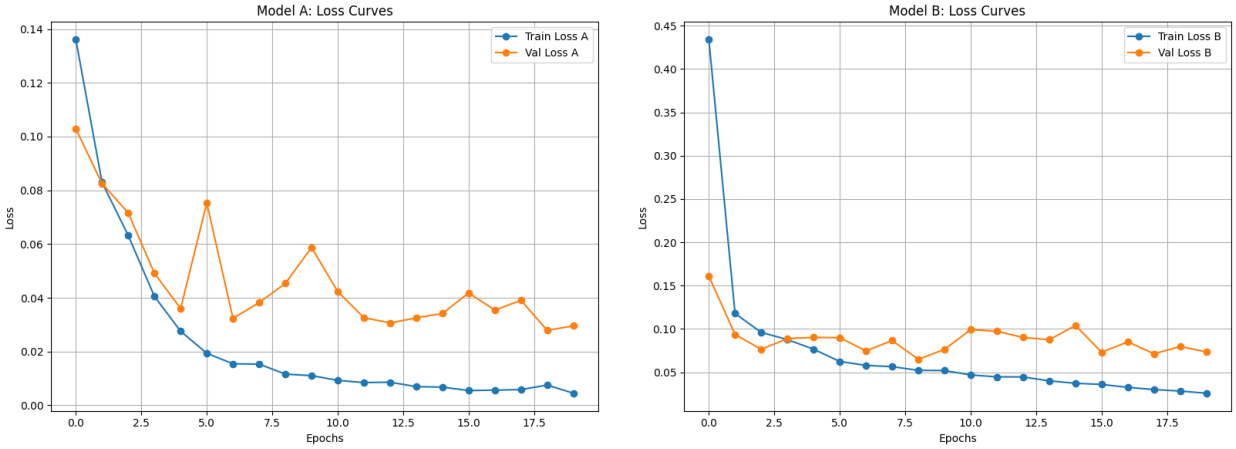



**Figure 7:** Validation accuracy for the five optimizers. Only Adam leaves chance level (20%).

## 5.2 Loss and accuracy curves




**(a)** Model A **(b)** Model B

**Figure 8:** Training and validation loss over 20 epochs.

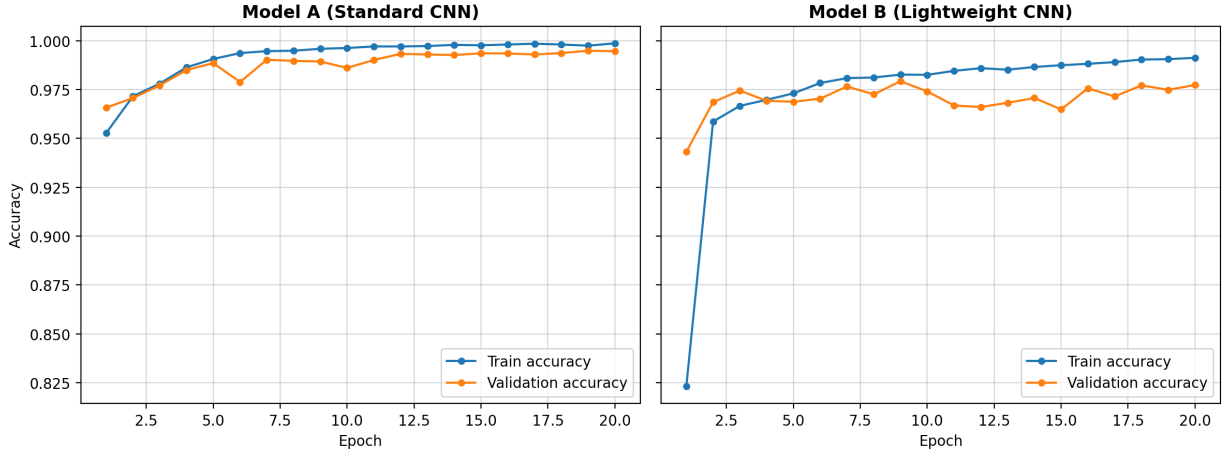



**Figure 9:** Training and validation accuracy for Model A and Model B (re-plotted from the per-epoch values logged by `model.fit`; the notebook itself plots only the loss curves).

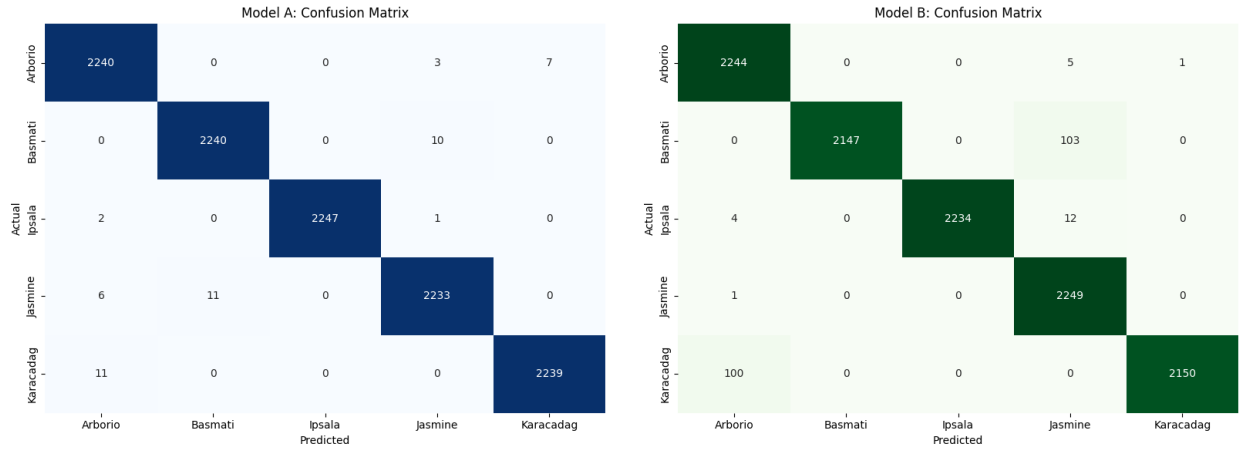
**Model A.** Training loss falls steadily to 0.0045 (99.86% training accuracy). Validation loss follows it down to about 0.03, with spikes at epochs 6 (0.075) and 10 (0.059), and ends at 0.0296 (99.46% validation accuracy). The gap between training and validation loss widens over the run (0.0045 vs. 0.0296 at epoch 20) – mild over-fitting – but validation loss does *not* diverge upwards, and its minimum (0.0278) occurs at epoch 19.

**Model B.** Training loss decreases smoothly to 0.0258 (99.12% training accuracy). Validation loss reaches its minimum (0.0650, 97.92% accuracy) at epoch 9 and afterwards fluctuates between about 0.07 and 0.10 instead of continuing to fall; it ends at 0.0735 (97.73% accuracy). Model B therefore also shows a generalisation gap after roughly epoch 9, and in absolute terms its loss gap (0.048) is larger than Model A's (0.025). The idea that a small parameter count acts as a natural regulariser is *not* supported by this run: the limiting factor for Model B appears to be representational capacity at 64×64 resolution, not over-fitting alone (see the confusion analysis below).

Validation and test accuracy agree closely (A: 99.46% vs. 99.55%; B: 97.73% vs. 97.99%), so the test results are not inflated by selecting on the validation set.

### 5.3 Test-set evaluation

Both models were evaluated on the 11,250 held-out test images (2,250 per class).




**(a)** Model A **(b)** Model B

**Figure 10:** Confusion matrices on the test set (rows: actual class, columns: predicted class).

| | **Model A (Standard CNN)** | | | **Model B (Lightweight CNN)** | | |
|---|---|---|---|---|---|---|
| **Class** | Precision | Recall | F1 | Precision | Recall | F1 |
| Arborio | 99.16 | 99.56 | 99.36 | 95.53 | 99.73 | 97.59 |
| Basmati | 99.51 | 99.56 | 99.53 | 100.00 | 95.42 | 97.66 |
| Ipsala | 100.00 | 99.87 | 99.93 | 100.00 | 99.29 | 99.64 |
| Jasmine | 99.38 | 99.24 | 99.31 | 94.93 | 99.96 | 97.38 |
| Karacadag | 99.69 | 99.51 | 99.60 | 99.95 | 95.56 | 97.71 |
| Macro average | 99.55 | 99.55 | 99.55 | 98.08 | 97.99 | 97.99 |
| **Accuracy** | **99.55%** (51 errors) | | | **97.99%** (226 errors) | | |

**Table 6:** Per-class test metrics (%), computed from the confusion matrices in Figure 10; macro precision and recall match the scikit-learn report in the notebook.

**Confusion analysis.** Model A's 51 errors are spread thinly over several pairs (largest: 11 Jasmine→Basmati, 11 Karacadag→Arborio, 10 Basmati→Jasmine). Model B's 226 errors are highly concentrated: **103 Basmati→Jasmine and 100 Karacadag→Arborio account for 90% of them**, while the other classes are almost perfect (Ipsala 2,234/2,250 correct; Jasmine 2,249/2,250). Both confused pairs are visually similar in shape – Basmati and Jasmine are both elongated, Karacadag and Arborio are both short and rounded (Figure 2) – and the confusions are one-directional (Basmati is mistaken for Jasmine, but Jasmine is hardly ever mistaken for Basmati), meaning that the network has learnt a decision boundary biased towards Jasmine and Arborio. A plausible cause is that fine texture and size cues are largely lost at $64 \times 64$ and that the narrow 128–256-channel separable stack has less capacity to recover them; we did not run an ablation to confirm this. For the sorting application the practical implication is that Model B's errors would be systematic (Basmati lots contaminated with Jasmine) rather than random.

### 5.4 Model comparison: standard vs. depthwise-separable

| Metric | Model A (standard) | Model B (separable) |
|---|---|---|
| Total trainable parameters | 1,142,597 | 79,168 ( $<$ 100k ) |
| Size of saved `.h5` file | 13,439.88 KB (13.12 MB) | 994.44 KB (0.97 MB) |
| Float32 weights only (params $\times$ 4 B) | 4,463 KB (4.36 MB) | 309 KB (0.30 MB) |
| Forward-pass MACs (approx.) | 42.6 M | 7.6 M |
| Training time per epoch (Colab GPU) | 31.98 s | 34.66 s |
| Test accuracy | **99.55%** | 97.99% |
| Macro precision / recall / F1 | 99.55 / 99.55 / 99.55% | 98.08 / 97.99 / 97.99% |

**Table 7:** Model A vs. Model B. MB = $1024^2$ bytes, KB = 1024 bytes.

**Note on file sizes.** The sizes in the notebook come from `model.save('*.h5')`, which stores the Adam optimizer's two moment tensors in addition to the weights. This inflates every file by roughly 3$\times$ (A: 4.36 MB weights vs. 13.12 MB file; B: 0.30 MB vs. 0.97 MB). The deployed model needs only the weights, so the third row is the more relevant figure for flash memory; we keep the file sizes because they are what the notebook measured and what the assignment's "size on disk" asks for.

**Trade-offs observed.**

- **Memory:** Model B is 93% smaller in parameters and 92.6% smaller on disk (994 KB vs. 13,440 KB), and fits comfortably under 1 MB.
- **Compute:** 5.6$\times$ fewer MACs (Section 3.4), per-layer reductions of 7–8$\times$.
- **Accuracy:** Model B loses 1.56 percentage points (97.99% vs. 99.55%), i.e. it makes 4.4$\times$ as many errors. Whether this is acceptable depends on the cost of a mis-sorted grain.
- **Training time:** essentially equal. Despite having 5.6$\times$ fewer MACs, Model B trained slightly *slower* per epoch (34.7 s vs. 32.0 s). Per-epoch times in the logs vary between 26 and 43 s for identical workloads (shared Colab GPU), so a 2.7 s difference is within noise and we draw no conclusion from it other than that fewer MACs did not translate into faster GPU training. Depthwise convolutions have low arithmetic intensity and are commonly memory-bound on GPUs, which is a likely contributor, though we did not profile it.

## 6 Comparison with Lightweight State-of-the-Art Networks

### 6.1 Transfer-learning set-up

We used two ImageNet pre-trained networks designed for mobile/edge deployment [6, 7]:

- **MobileNetV2** – inverted residual blocks with linear bottlenecks built on depthwise-separable convolutions;
- **EfficientNetB0** – compound scaling of depth, width and resolution (mobile inverted-bottleneck blocks).

For each network the convolutional base is **frozen**, followed by global average pooling, a 128-unit ReLU dense layer and a 5-way softmax; only this head is trained (Adam, $\eta = 10^{-3}$, 15 epochs, same data splits and batch size as the custom models). MobileNetV2 receives $[-1, 1]$ inputs via a `Rescaling` layer; EfficientNetB0 receives raw $[0, 255]$ pixels. Both bases were instantiated with a $64 \times 64 \times 3$ input; Keras warns that the available ImageNet weights are the $224 \times 224$ ones, which are then applied to inputs 3.5$\times$ smaller than those they were trained on.

Table 8 gives parameter counts (obtained by instantiating the same architectures in Keras; the

notebook only stated approximate values). Because the base is frozen, only 164,613 parameters of each SOTA model are trainable – still twice as many as Model B's *total*.

| Model | Total params | Trainable | Frozen | Fp32 weights |
|---|---|---|---|---|
| MobileNetV2 + head | 2,422,597 | 164,613 | 2,257,984 | 9.24 MB |
| EfficientNetB0 + head | 4,214,184 | 164,613 | 4,049,571 | 16.08 MB |

**Table 8:** Parameter counts of the fine-tuned SOTA models.

## 6.2 Training dynamics

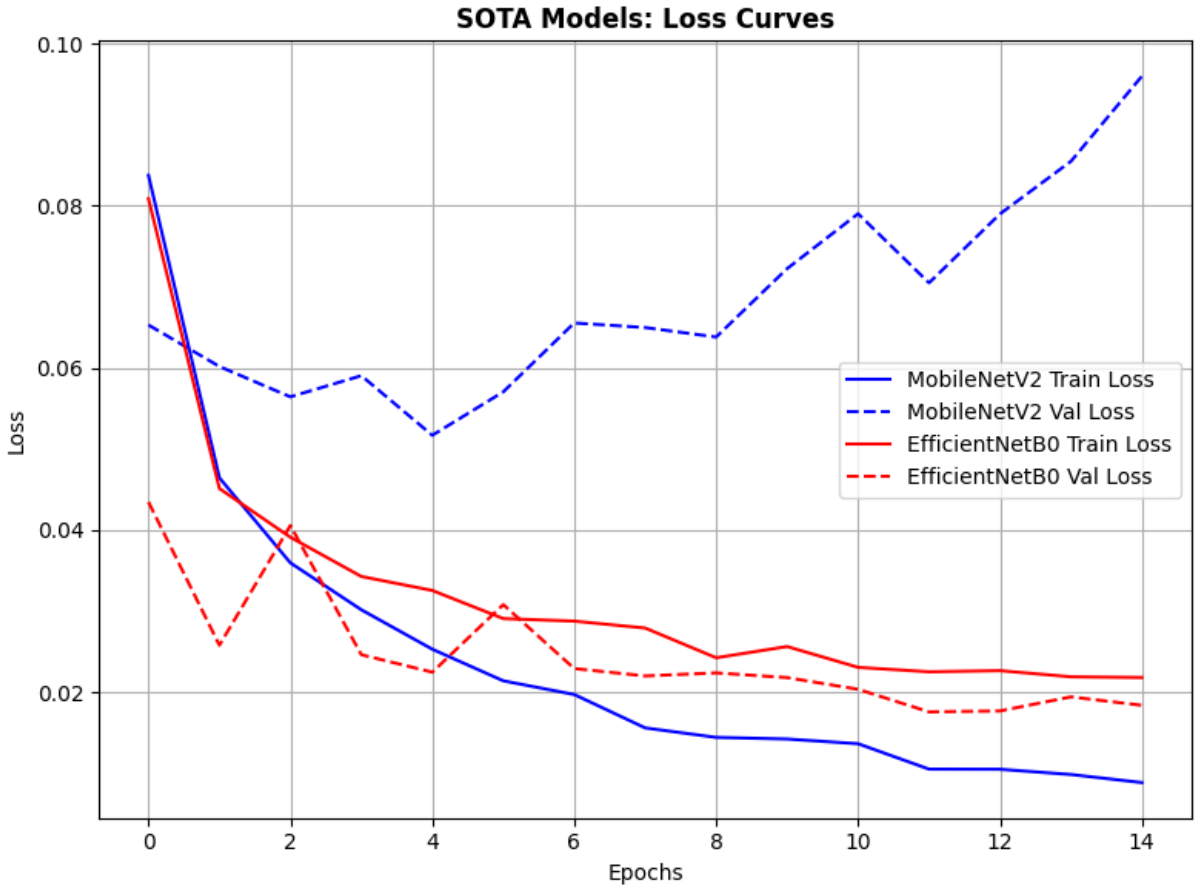



**Figure 11:** Training (solid) and validation (dashed) loss of the fine-tuned SOTA models.

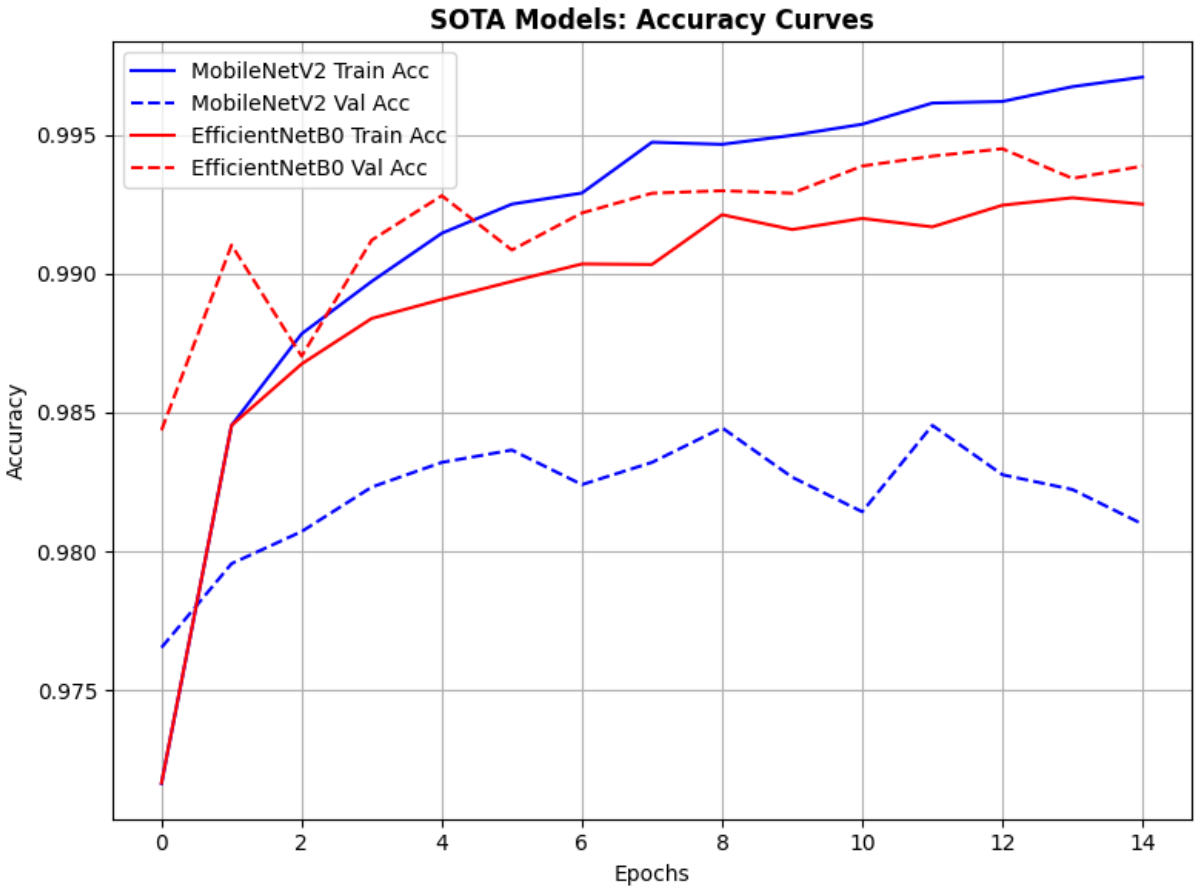



**Figure 12:** Training (solid) and validation (dashed) accuracy of the fine-tuned SOTA models.

Both models start from high accuracy (97.2% training accuracy in epoch 1) because the pre-trained features already separate the grains. EfficientNetB0 trains stably: validation loss falls to 0.018 and validation accuracy reaches 99.4%. MobileNetV2 shows clear over-fitting of its head: training loss keeps falling (0.0089) while validation loss rises from its minimum of 0.052 (epoch 5) to 0.096 (epoch 15), and validation accuracy stays at about 98%. Early stopping at about epoch 5 would have been appropriate; the reported results are from the final epoch.

### 6.3 Test-set results

| | **MobileNetV2** | | | **EfficientNetB0** | | |
|---|---|---|---|---|---|---|
| **Class** | P | R | F1 | P | R | F1 |
| Arborio | 0.99 | 0.97 | 0.98 | 0.99 | 0.99 | 0.99 |
| Basmati | 0.97 | 0.99 | 0.98 | 1.00 | 0.99 | 1.00 |
| Ipsala | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| Jasmine | 0.99 | 0.97 | 0.98 | 0.99 | 1.00 | 0.99 |
| Karacadag | 0.98 | 0.99 | 0.98 | 1.00 | 0.99 | 0.99 |
| Macro avg | 0.98 | 0.98 | 0.98 | 0.99 | 0.99 | 0.99 |
| **Accuracy** | **98.41%** | | | **99.45%** | | |

**Table 9:** Per-class test metrics of the SOTA models (scikit-learn report, 2 decimals; 2,250 test images per class). Macro precision/recall: MobileNetV2 98.42/98.41%, EfficientNetB0 99.45/99.45%. Saved-file sizes: 10.85 MB and 17.79 MB.

# 7 Final Comparison and Trade-off Discussion

| Metric | Model A | Model B | MobileNetV2 | EfficientNetB0 |
|---|---|---|---|---|
| Build approach | scratch | scratch | transfer | transfer |
| Epochs trained | 20 | 20 | 15 | 15 |
| Total parameters | 1.14 M | **79,168** | 2.42 M | 4.21 M |
| Trainable parameters | 1.14 M | 79,168 | 164,613 | 164,613 |
| Saved file (`.h5`) | 13.12 MB | **0.97 MB** | 10.85 MB | 17.79 MB |
| Fp32 weights only | 4.36 MB | **0.30 MB** | 9.24 MB | 16.08 MB |
| MACs per image (approx.) | 42.6 M | **7.6 M** | 25.2 M | 33.5 M |
| Time per epoch | **31.98 s** | 34.66 s | 34.57 s | 39.84 s |
| Test accuracy | 99.55% | 97.99% | 98.41% | 99.45% |
| Macro precision / recall | 99.55 / 99.55 | 98.08 / 97.99 | 98.42 / 98.41 | 99.45 / 99.45 |

**Table 10:** Final comparison on the 11,250-image test set. MB = $1024^2$ bytes.

## 7.1 Accuracy versus memory footprint

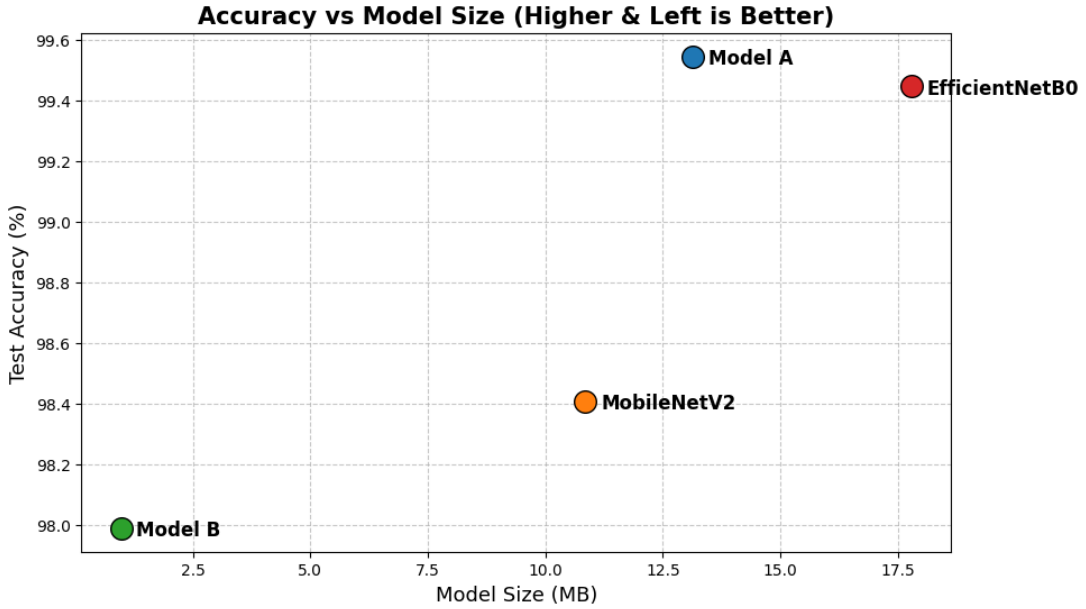



**Figure 13:** Test accuracy against saved model size (higher and further left is better). The $x$-axis shows the `.h5` file sizes (MB = $1024^2$ bytes).

EfficientNetB0 and Model A sit at the top of the accuracy range (99.45% and 99.55%); Model B is 1.46 points below EfficientNetB0 and 0.42 points below MobileNetV2 (98.41%), while its saved file is 94.5% smaller than EfficientNetB0's and 91.1% smaller than MobileNetV2's (98% and 97% smaller in float32 weights alone). Two observations stand out:

1. **Transfer learning gave no accuracy advantage on this task.** Model A, trained from scratch with 1.14 M parameters, beats MobileNetV2 and matches EfficientNetB0 at roughly half to a quarter of their parameter count. The grains are photographed on a uniform background, so low-level ImageNet features add little, and our SOTA runs trained only the head on a frozen backbone fed with inputs far smaller than the 224×224 it was pre-trained for. Unfreezing the upper blocks might raise the SOTA scores; we did not test it.

2. **Model B is the only model on the memory-efficient side of the plot**, and the accuracy it trades for that is small in absolute terms but not negligible in error terms (4.4× Model A's error count).

## 7.2 Computational cost and latency

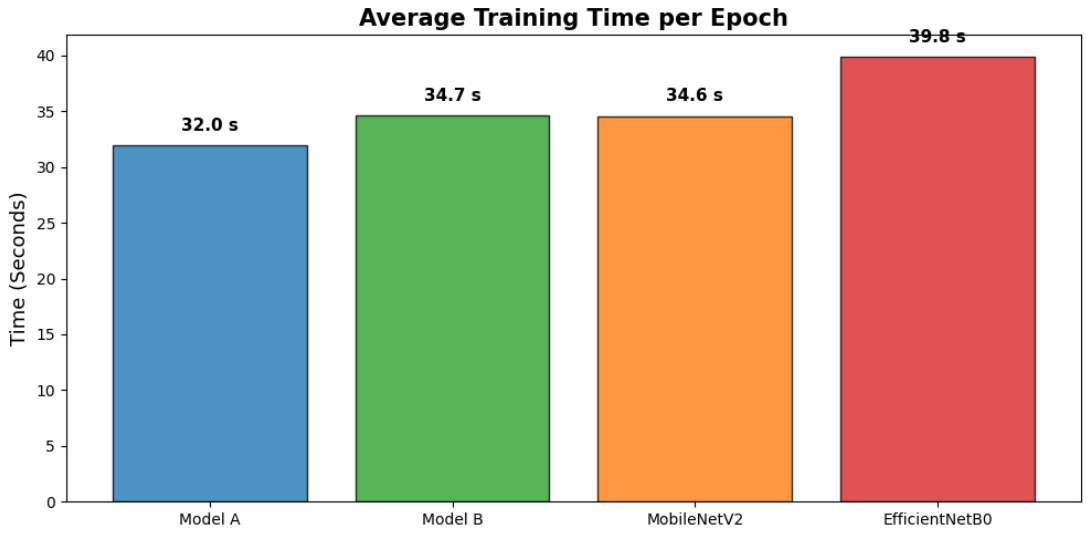



**Figure 14:** Average training time per epoch (Colab GPU). This measures training on a shared cloud GPU, not inference on an edge device.

Training time (Figure 14) is a poor proxy for deployment cost: the four models differ by less than 8 s per epoch, with Model A the fastest and EfficientNetB0 the slowest, and the differences are comparable to the run-to-run variation noted above. For deployment, the relevant quantities are MACs and memory (Figure 5). Model B needs 5.6× fewer MACs than Model A, 3.3× fewer than MobileNetV2 and 4.4× fewer than EfficientNetB0. Because depthwise layers are memory-bound, real latency gains on a given chip may be smaller than the MAC ratios suggest; **inference latency was not measured in this study** and should be benchmarked on the target hardware (e.g. TensorFlow Lite for Microcontrollers).

## 7.3 Activation memory: weights are not the whole story

Peak RAM during inference is dominated by feature maps, not weights. In Model B (and Model A) the largest live tensors occur in the first layer: the $64 \times 64 \times 32$ output (131,072 values) plus the $64 \times 64 \times 3$ input (12,288 values), about 143,000 values. That is about 560 KB in float32 but about 140 KB in INT8 (analytic estimate, excluding interpreter overhead). So the claim that Model B is "deployable on a microcontroller" really holds for parts with a few hundred KB of SRAM and only after INT8 quantisation, not for very small boards. Reducing the first-layer width or using a stride-2 first convolution would lower this peak.

### 7.4 Advantages and limitations of a custom sub-100k model

| Advantages (Model B) | Limitations (Model B) |
|---|---|
| Very small: 79,168 parameters, 0.30 MB in float32 (about 77 KB after INT8 quantisation) fits flash of low-cost MCUs. | 1.5–1.6 points lower accuracy than the best models; errors are systematic (Basmati→Jasmine, Karacadag→Arborio). |
| Fewest MACs (7.6 M): lowest energy and latency potential per frame. | Latency gain is smaller than MAC ratio on memory-bound hardware; not measured here. |
| Architecture and training are fully under our control; no dependency on ImageNet weights, input-size or licence constraints. | Trained from scratch: needs hyper-parameter work, and shows a validation plateau after epoch 9; no pre-trained features to fall back on for harder data. |
| Fully-convolutional + GAP head is independent of input size and adds no large dense layer. | Peak activation memory (about 140 KB INT8) is the same as the much larger Model A. |

**Table 11:** Custom lightweight model: advantages and limitations.

**Pre-trained SOTA models** converge in a few epochs and generalise well (99.45% for EfficientNetB0), and would be the better choice on a Raspberry Pi-class device with hundreds of MB of RAM; but at 9–16 MB of float32 weights they cannot be flashed onto small microcontrollers, and their head-only training offers no advantage here over Model A.

**Recommendation.** For the target sorting machine: (i) on a Raspberry Pi-class node, Model A or EfficientNetB0 (99.5%); (ii) on a microcontroller, Model B after INT8 quantisation, accepting about 2% error, ideally with a confidence-threshold reject option for the two weak class pairs.

## 8 Limitations and Threats to Validity

- **Single run, single seed (42).** Differences of a few tenths of a percentage point (e.g. Model A 99.55% vs. EfficientNetB0 99.45%, a gap of 11 images in 11,250) are within what a different seed could change; no confidence intervals were computed.
- **Optimizer study used one learning rate**, and the same epoch budget (15) as the SOTA models rather than a tuned schedule (Section 4).
- **SOTA models were not truly fine-tuned:** the base was frozen, MobileNetV2 was not early-stopped, and ImageNet weights were applied at $64 \times 64$.
- **Timing noise:** epoch times on a shared Colab GPU vary by up to 15 s for identical workloads; we report means of single runs.
- **Saved sizes include Adam state**; float32 weight sizes are given alongside.
- **No on-device inference benchmark**, and no confusion matrices for the SOTA models (only scikit-learn reports).
- **Dataset simplicity:** uniform black background and a single grain per image make the task easier than real sorting-line conditions (lighting changes, overlapping grains, dust); accuracies here will not transfer directly.

## 9 Conclusion and Future Work

We designed a standard CNN (Model A, 1,142,597 parameters) and a depthwise-separable CNN (Model B, 79,168 parameters) and evaluated both against fine-tuned MobileNetV2 and EfficientNetB0 on the full 75,000-image Rice dataset at $64 \times 64$. Model A (99.55%) and EfficientNetB0

(99.45%) were most accurate; MobileNetV2 reached 98.41%. Model B achieved **97.99%** while satisfying the <100,000-parameter constraint, using 14× fewer parameters than Model A, 30–53× fewer than the SOTA models, 3.3–5.6× fewer MACs, and a 0.30 MB (float32) weight footprint. Its accuracy loss is concentrated in two confusable class pairs. Adam was the only optimizer that trained at the shared learning rate of $10^{-3}$. Training time was not a differentiator.

**Future work.**

1. **Post-training INT8 quantisation** (and quantisation-aware training) of Model B, which should reduce its weights to about 77 KB; verify the accuracy cost and run on-device latency/RAM benchmarks.
2. **Learning-rate and momentum sweep** for SGD, and learning-rate decay for Adam.
3. **Targeted improvements for Model B**: wider early layers, augmentation (rotation, brightness), residual connections, or a slightly higher input resolution to address the Basmati/Jasmine and Karacadag/Arborio confusions.
4. **Unfreezing the top blocks** of the SOTA models, early stopping, and repeated runs over several seeds.

## 10 Source Code and Reproducibility

The notebook (data download, splitting, both models, the optimizer study, SOTA fine-tuning and every figure in this report), the LaTeX source and the result images are available at:
https://github.com/senuja-dilmith/EN3150_A03_GalTricals

All Python, NumPy and TensorFlow seeds are set by a `set_global_deterministic_seeds(42)` function before the dataset is built, and the stratified splits use `random_state=42`. Exact bit-for-bit reproducibility of GPU training is not guaranteed across hardware or library versions.

## References

[1] S. K. Perera, *EN3150 Pattern Recognition – Lecture 06: Deep Neural Networks*, University of Moratuwa, 2026.

[2] S. K. Perera, *EN3150 Pattern Recognition – Lecture 07: Convolutional Neural Networks*, University of Moratuwa, 2026.

[3] M. Koklu, *Rice Image Dataset*, Kaggle. https://www.kaggle.com/datasets/muratkokludataset/rice-image-dataset

[4] I. Cinar and M. Koklu, "Classification of Rice Varieties Using Artificial Intelligence Methods," *Int. J. Intelligent Systems and Applications in Engineering*, vol. 7, no. 3, pp. 188–194, 2019.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018, pp. 4510–4520.

[7] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019, pp. 6105–6114.

[8] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," *CVPR*, 2017, pp. 1251–1258.

[9] F. Chollet et al., *Keras API reference: SeparableConv2D*. https://keras.io/api/layers/convolution_layers/separable_convolution2d/